L10 - L11 – Scikit-learn

Data Mining

Dr Andrea Visentin

scikits learn

machine learning in Python

Scikit-learn

- Simple and efficient tools for predictive data analysis
- Accessible to everybody, and reusable in various contexts
- Built on NumPy, SciPy, and matplotlib
- Open source, commercially usable - BSD license

- https://scikit-learn.org/stable/index.html

Iris dataset

```
from sklearn import datasets

iris = datasets.load_iris()
```

KNN in scikit-learn

```
from sklearn import datasets
from sklearn import neighbors

iris = datasets.load_iris()


knn = neighbors.KNeighborsClassifier(n_neighbors = 5)

knn.fit(iris.data, iris.target)

print(knn.predict([[3, 5, 4, 2]]))
```

External Dataset

In the following example, we will read a dataset from an external resource and we will try to apply the techniques we learned.

Data

Label

Wine dataset

We will use the wine dataset.

A well known and easy dataset.

You can find it at:

http://archive.ics.uci.edu/ml/datasets/Wine

Data Set Characteristics:	Multivariate	**Number of Instances:**	178	**Area:**	Physical
Attribute Characteristics:	Integer, Real	**Number of Attributes:**	13	**Date Donated**	1991-07-01
Associated Tasks:	Classification	**Missing Values?**	No	**Number of Web Hits:**	1850558

Load the data

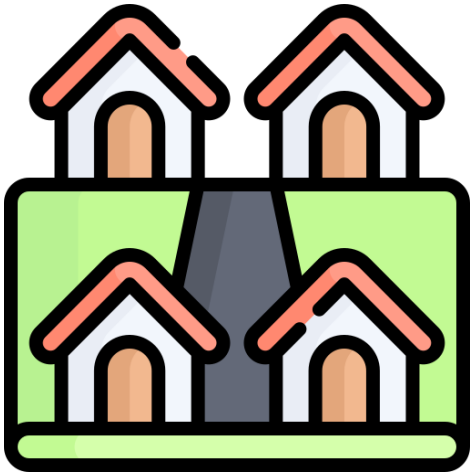
Load and prepare the dataset

Load the data

```
import pandas as pd

df = pd.read_csv("http://archive.ics.uci.edu/ml/machine-learning-databases/wine/wine.data", header = None)

df.columns = ["Class", "Alcohol", "Malic acid", "Ash" , "Alcalinity of ash", "Magnesium", "Total phenols", "Flavanoids", "Nonflavanoid phenols", "Proanthocyanins", "Color intensity", "Hue", "OD280/OD315 of diluted wines", "Proline"]
```

kNN in Scikit-learn

- Classifier implementing the k-nearest neighbours vote.
- https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsClassifier.html



Train and test a model

```
from sklearn import neighbors

knn = neighbors.KNeighborsClassifier(n_neighbors = 5)

knn.fit(df.loc[:,"Alcohol":"Proline"], df["Class"])

knn.predict(df.loc[1:4,"Alcohol":"Proline"])
```

Train and test a model

```
from sklearn import neighbors

knn = neighbors.KNeighborsClassifier(n_neighbors = 5)

features = df.loc[:,"Alcohol":"Proline"]
labels = df["Class"]

knn.fit(features,labels)

knn.predict(features[1:23])
```

Check the accuracy

To test how well our approach is doing, we need to compute the accuracy!

Classification Accuracy

- Most used and simple way to evaluate the performance of a classifier.

$$Accuracy = \frac{\#\ istances\ classified\ correctly}{\#\ instances}$$

Classification Accuracy

Predicted

Actual		Class A	Class B
	Class A	True Positives	False Negatives
	Class B	False Positives	True Negatives

- The correct way to measure accuracy is:

$$Accuracy = \frac{True\ Positive + True\ Negative}{True\ Positive + False\ Negative + False\ Positive + True\ Negatives}$$

Check the accuracy

What is the baseline we should check?

Check a model accuracy

```
from sklearn import neighbors
from sklearn import metrics

knn = neighbors.KNeighborsClassifier(n_neighbors = 5)

features = df.loc[:,"Alcohol":"Proline"]
labels = df["Class"]

# Train the KNN
knn.fit(features,labels)

# Predict the classes
results = knn.predict(features)

# Compute the accuracy
print(metrics.accuracy_score(results, labels))
```

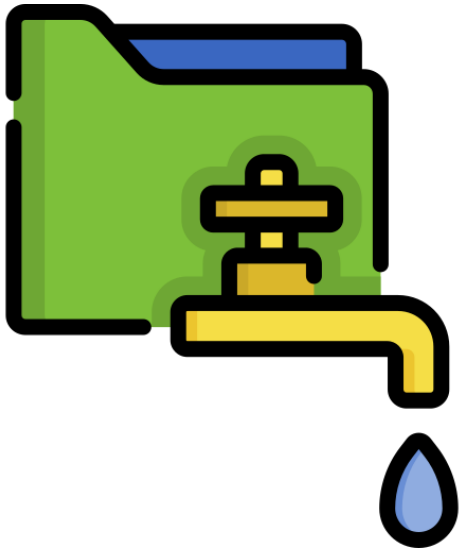
How can we create the perfect classifier?

Can we reach 100% accuracy?

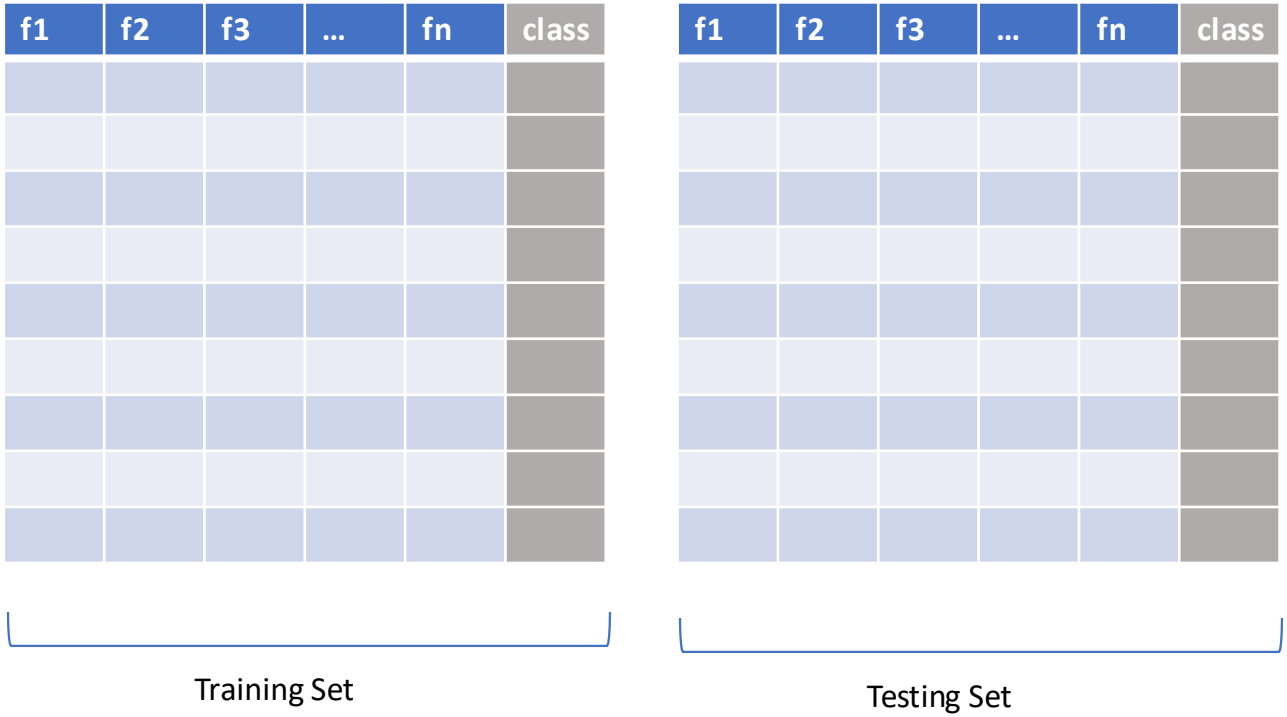
What should we do?

Data Leakage

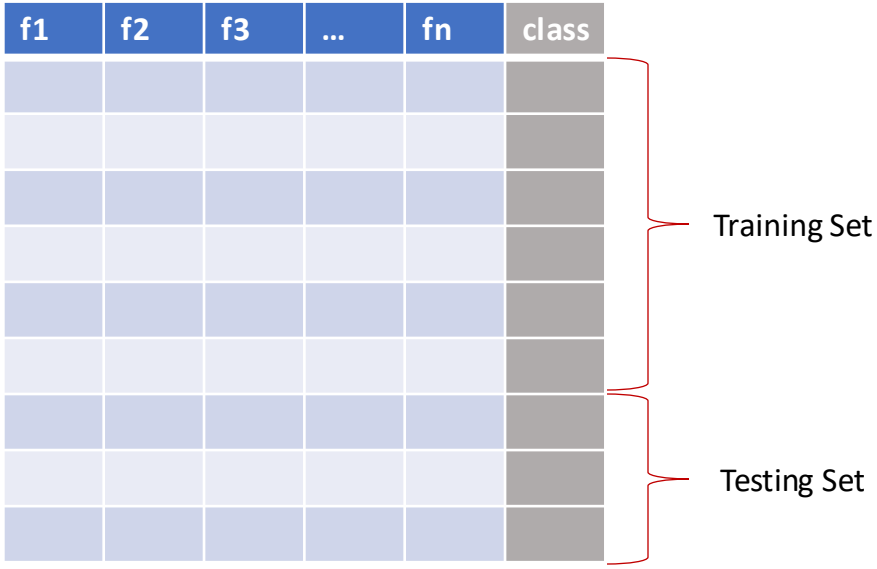
- Two main causes:
  - You test your model on the same data that it is trained on
  - Your data already contains the answer you are looking for



Holdout Set Configuration



Holdout Set Configuration



Train Test Split

- In scikit-learn a random split into training and test sets can be quickly computed with the train_test_split helper function.
- As arguments we pass it the **original data** and **target** as well as the **percentage of the original data** we want for the training data. We also pass it a random seed.

Train Test Split

```
from sklearn import model_selection


train_features, test_features, train_labels, test_labels
= model_selection.train_test_split(features, labels,
test_size=0.2, random_state=0)

print(train_features.shape, train_labels.shape)
print(test_features.shape, test_labels.shape)
```

Train Test Split

```
from sklearn import neighbors
from sklearn import metrics
from sklearn import model_selection

knn = neighbors.KNeighborsClassifier(n_neighbors = 15)

features = df.loc[:,"Alcohol":"Proline"]
labels = df["Class"]

# Train - Test Split
train_features, test_features, train_labels, test_labels =
model_selection.train_test_split(features, labels, test_size=0.2, random_state=0)

# Train the KNN
knn.fit(train_features,train_labels)

# Predict the classes
results = knn.predict(test_features)

# Compute the accuracy
print(metrics.accuracy_score(results, test_labels))
```

Train Test Split

```
from sklearn import neighbors
from sklearn import metrics
from sklearn import model_selection

knn = neighbors.KNeighborsClassifier(n_neighbors = 15)

features = df.loc[:,"Alcohol":"Proline"]
labels = df["Class"]

# Train - Test Split
train_features, test_features, train_labels, test_labels =
model_selection.train_test_split(features, labels, test_size=0.2, random_state=0)

# Train the KNN
knn.fit(train_features,train_labels)

# Predict and check the accuracy (shortcut)
print(knn.score(test_features, test_labels))
```

Confusion Matrix

Confusion matrix contains information about the actual and predicted classifications of a classification algorithm

- **True positives (TP)** - Number of instances of Category A that were correctly classified as Category A.
- **False Positives (FP)** - Number of instances of Category B that were incorrectly classified as Category A.
- **False Negative (FN)** - Number of instances of Category A that were incorrectly classified as Category B.
- **True Negative (TN)** - Number of instances of Category B that were correctly classified as Category B.

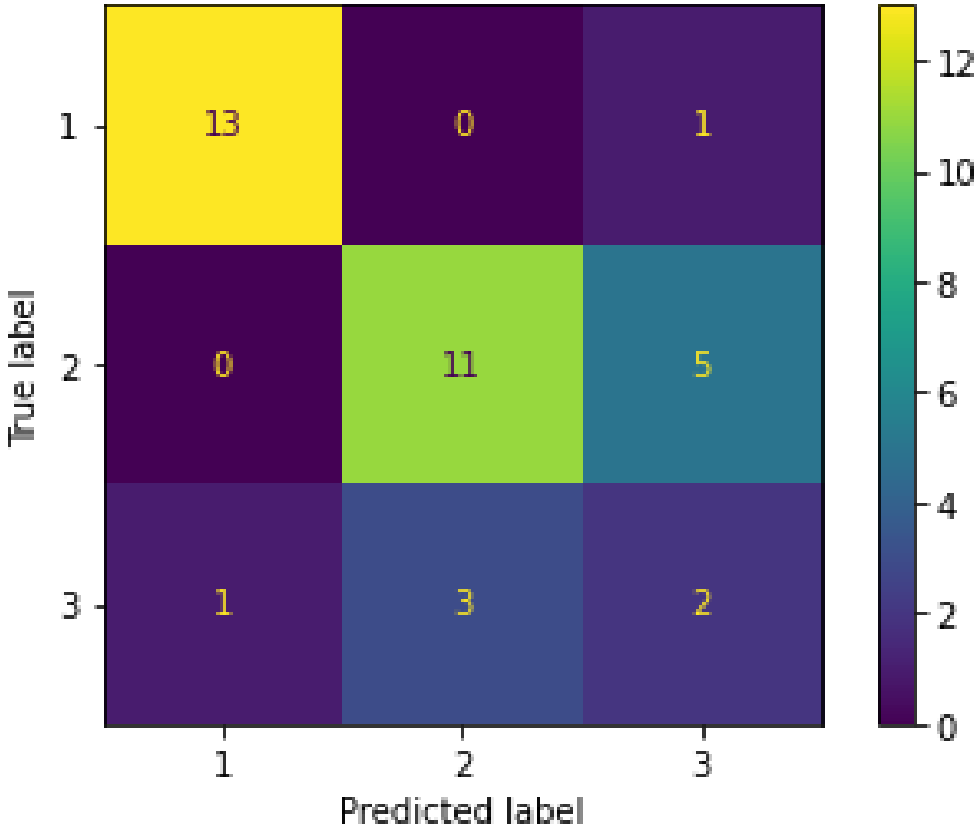
		Predicted	
		Category A	Category B
Actual	Category A	*w*	*x*
	Category B	*y*	*z*

Confusion Matrix print

```
from sklearn import neighbors

from sklearn import metrics

from sklearn import model_selection

import matplotlib.pyplot as plt

knn = neighbors.KNeighborsClassifier(n_neighbors = 15)

features = df.loc[:,"Alcohol":"Proline"]

labels = df["Class"]

# Train - Test Split

train_features, test_features, train_labels, test_labels = model_selection.train_test_split(features, labels, test_size=0.2, random_state=0)

# Train the KNN

knn.fit(train_features,train_labels)

predictions = knn.predict(test_features)

# Confusion Matrix print

cm = metrics.confusion_matrix(test_labels, predictions, labels = knn.classes_)

disp = metrics.ConfusionMatrixDisplay(confusion_matrix=cm,

display_labels=knn.classes_)

disp.plot()
plt.show()
```

Confusion Matrix print

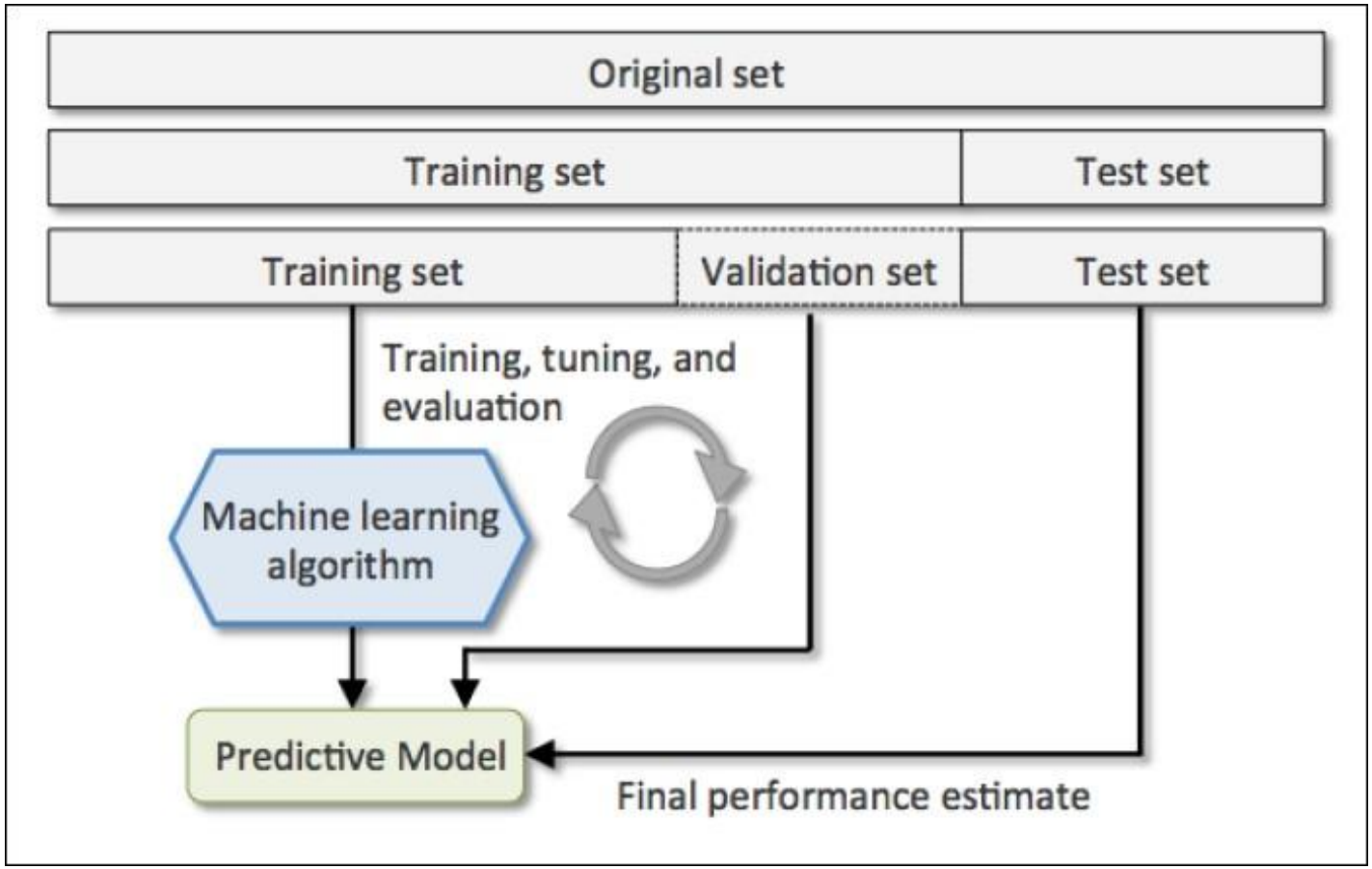
```
from sklearn import neighbc
from sklearn import metrics
from sklearn import model_s
import matplotlib.pyplot as

knn = neighbors.KNeighborsC
features = df.loc[:,"Alcohc
labels = df["Class"]
# Train - Test Split
train_features, test_featur
# Train the KNN
knn.fit(train_features,trai

predictions = knn.predict(t
# Confusion Matrix print
cm = metrics.confusion_matr
disp = metrics.ConfusionMat
display_labels=knn.classes_

disp.plot()
plt.show()
```




```
ize=0.2, random_state=0)
```

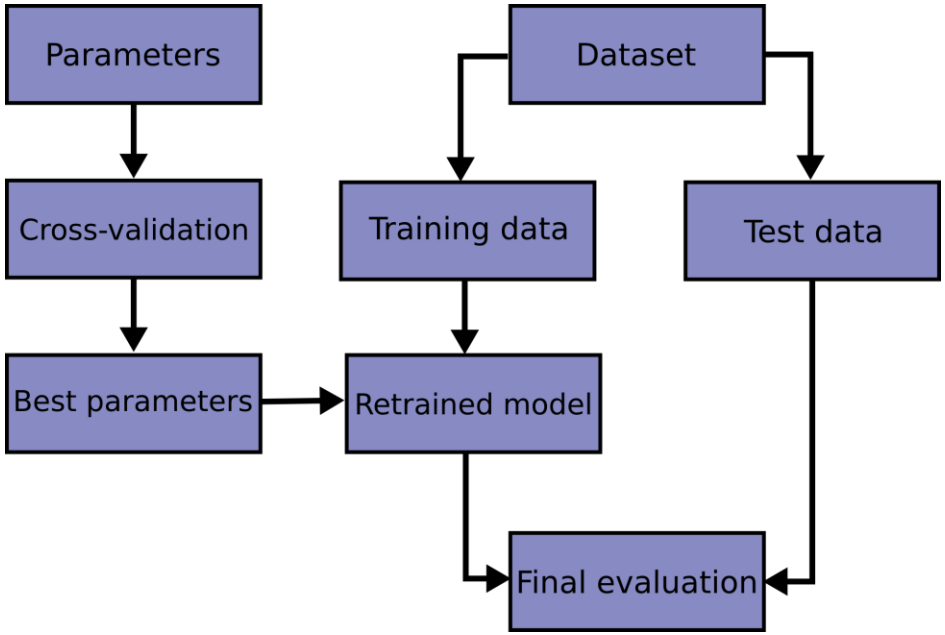
How should we select the best k?

Holdout Cross-Validation



Cross-Validation

- https://scikit-learn.org/stable/modules/cross_validation.html



Cross-Validation

```
from sklearn import neighbors
from sklearn import metrics
from sklearn import model_selection
import matplotlib.pyplot as plt

knn = neighbors.KNeighborsClassifier(n_neighbors = 3)

features = df.loc[:,"Alcohol":"Proline"]
labels = df["Class"]

# Train - Test Split
train_features, test_features, train_labels, test_labels =
model_selection.train_test_split(features, labels, test_size=0.2, random_state=999)

# Cross validation of the metrics
scores = model_selection.cross_val_score(knn, train_features, train_labels, cv=10)
print(scores)

# Print mean and sd
print("%0.2f accuracy with a standard deviation of %0.2f" % (scores.mean(), scores.std()))
```

Cross-Validation – Final Test

```
# Test on the unseen testset
knn.fit(train_features, train_labels)
knn.score(test_features, test_labels)
```

Cross-Validation – Grid Search

```
from sklearn import neighbors
from sklearn import metrics
from sklearn import model_selection
import matplotlib.pyplot as plt

knn = neighbors.KNeighborsClassifier()
parameters = {'n_neighbors': [1, 3, 5, 7, 11]}

features = df.loc[:,"Alcohol":"Proline"]
labels = df["Class"]
train_features, test_features, train_labels, test_labels =
model_selection.train_test_split(features, labels, test_size=0.2, random_state=0)

clf = model_selection.GridSearchCV(knn, parameters)

clf.fit( train_features, train_labels)

print("The best classifier is:", clf.best_estimator_)
print("Its accuracy is:",clf.best_score_)
print("Its parameters are:",clf.best_params_)
```

How can we improve the accuracy?

Data Normalization

- Min-Max normalization
  - For an attribue *A* we have values in range $[min_A, max_A]$
  - We want to scale them in [0,1]

The new values can be computed as:

$$v' = \frac{v - min_A}{max_A - min_A}$$

MinMaxScaler

- Transform features by scaling each feature to a given range.
- This estimator scales and translates each feature individually such that it is in the given range on the training set, e.g. between zero and one.
- https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.MinMaxScaler.html#sklearn.preprocessing.MinMaxScaler

What is the error here?

```
from sklearn import preprocessing

train_features, test_features, train_labels, test_labels =
model_selection.train_test_split(features, labels, test_size=0.2,
random_state=0)

scaler = preprocessing.MinMaxScaler()
scaler.fit(train_features)


train_features = scaler.transform(train_features)

knn.fit(train_features, train_labels)

# Predict the classes
results = knn.predict(test_features)

# Compute the accuracy
print(metrics.accuracy_score(results, test_labels))
```

What is the accuracy now?

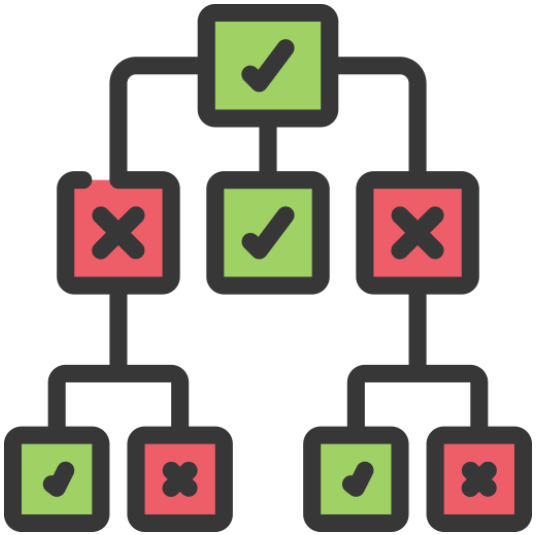
```
from sklearn import preprocessing

train_features, test_features, train_labels, test_labels =
model_selection.train_test_split(features, labels, test_size=0.2,
random_state=0)

scaler = preprocessing.MinMaxScaler()
scaler.fit(train_features)


train_features = scaler.transform(train_features)
test_features = scaler.transform(test_features)

knn.fit(train_features, train_labels)

# Predict the classes
results = knn.predict(test_features)

# Compute the accuracy
print(metrics.accuracy_score(results, test_labels))
```

What is the accuracy now?

```
from sklearn import preprocessing

train_features, test_features, train_labels, test_labels =
model_selection.train_test_split(features, labels, test_size=0.2, random_state=0)

scaler = preprocessing.MinMaxScaler()
scaler.fit(train_features)


train_features = scaler.transform(train_features)
test_features = scaler.transform(test_features)

knn.fit(train_features, train_labels)

# Predict the classes
results = knn.predict(test_features)

# Compute the accuracy
print(metrics.accuracy_score(results, test_labels))

print(sorted(scaler.data_min_))
print(sorted(scaler.data_max_))
```

Decision Tree in Scikit-learn

- Decision Trees (DTs) are a non-parametric supervised learning method used for classification and regression.
- https://scikit-learn.org/stable/modules/tree.html



Overview of ID3

Step 1

For all attributes that have not yet been used in the tree calculate their **entropy** and **information gain** values for the test samples

Step 2

Select the attribute that has the highest information gain

Step 3

Make a tree node containing that attribute

Decision Tree Parameters

- criterion
- splitter
- max_depth
- min_samples_split
- min_samples_leaf
- min_weight_fraction_leaf
- ecc

Decision Trees in Python

```
import pandas as pd
from sklearn import tree

df = pd.read_csv("http://archive.ics.uci.edu/ml/machine-learning-
databases/wine/wine.data", header = None)
df.columns = ["Class", "Alcohol", "Malic acid", "Ash" , "Alcalinity
of ash", "Magnesium", "Total phenols", "Flavanoids", "Nonflavanoid
phenols", "Proanthocyanins", "Color intensity", "Hue", "OD280/OD315
of diluted wines", "Proline"]

features = df.loc[:,"Alcohol": ]
labels = df['Class']

print(features.shape, labels.shape)

dtc = tree.DecisionTreeClassifier()
dtc = dtc.fit(features, labels)

dtc.score(features, labels)
```

Decision Trees in Python

```
from sklearn import model_selection

train_features, test_features, train_labels, test_labels
= model_selection.train_test_split(features, labels,
test_size=0.2, random_state=0)

dtc = dtc.fit(train_features, train_labels)

dtc.score(test_features, test_labels)
```

Should we normalize?

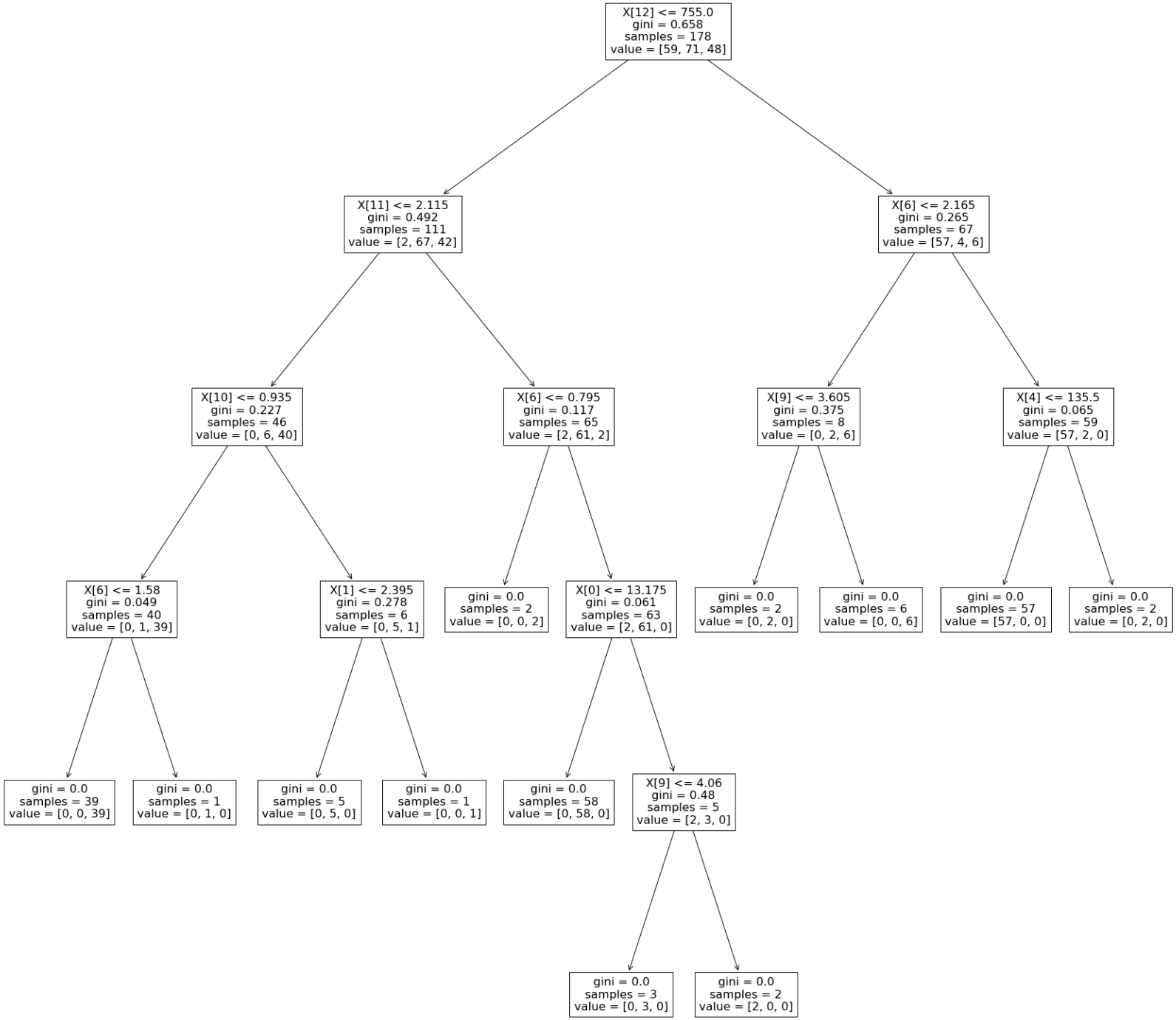
Plot Tree

- Plot a decision Tree

- https://scikit-learn.org/stable/modules/generated/sklearn.tree.plot_tree.html#sklearn.tree.plot_tree

Print Tree

```
import matplotlib.pyplot as plt

plt.figure(figsize=(30, 30)) # Resize figure
tree.plot_tree(dtc)
```


X[12] <= 755.0
gini = 0.658
samples = 178
value = [59, 71, 48]
X[11] <= 2.115
gini = 0.492
samples = 111
value = [2, 67, 42]
X[6] <= 2.165
gini = 0.265
samples = 67
value = [57, 4, 6]
X[10] <= 0.935
gini = 0.227
samples = 46
value = [0, 6, 40]
X[6] <= 0.795
gini = 0.117
samples = 65
value = [2, 61, 2]
X[9] <= 3.605
gini = 0.375
samples = 8
value = [0, 2, 6]
X[4] <= 135.5
gini = 0.065
samples = 59
value = [57, 2, 0]
X[6] <= 1.58
gini = 0.049
samples = 40
value = [0, 1, 39]
X[1] <= 2.395
gini = 0.278
samples = 6
value = [0, 5, 1]
gini = 0.0
samples = 2
value = [0, 0, 2]
X[0] <= 13.175
gini = 0.061
samples = 63
value = [2, 61, 0]
gini = 0.0
samples = 2
value = [0, 2, 0]
gini = 0.0
samples = 6
value = [0, 0, 6]
gini = 0.0
samples = 57
value = [57, 0, 0]
gini = 0.0
samples = 2
value = [0, 2, 0]
gini = 0.0
samples = 39
value = [0, 0, 39]
gini = 0.0
samples = 1
value = [0, 1, 0]
gini = 0.0
samples = 5
value = [0, 5, 0]
gini = 0.0
samples = 1
value = [0, 0, 1]
gini = 0.0
samples = 58
value = [0, 58, 0]
X[9] <= 4.06
gini = 0.48
samples = 5
value = [2, 3, 0]
gini = 0.0
samples = 3
value = [0, 3, 0]
gini = 0.0
samples = 2
value = [2, 0, 0]

Let’s focus on the Entropy

Entropy

Entropy 0 ("perfectly classified")
Entropy 1 ("totally random")

We can calculate the entropy of the dataset S as:

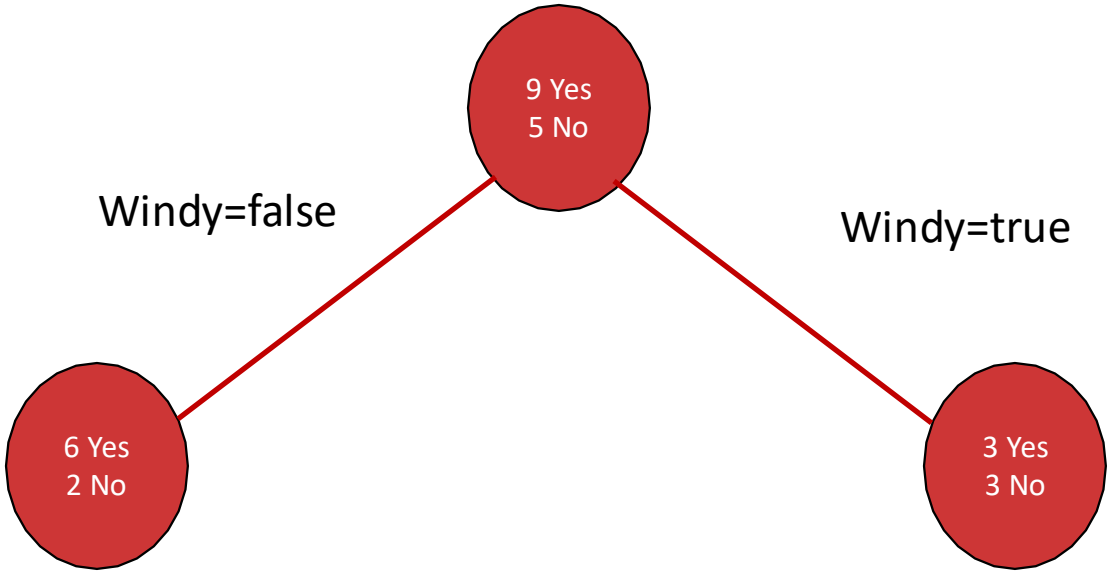
$$Ent(S) = H(S) = \sum_{i=1}^{n} -p_i \log_2 p_i$$

- For a dataset S with n different classes(outputs), proportion of class i is p_i (Number of instance of class i divided by total number of instances)

Note we can use the same formula for calculating the entropy for various attribute values such as Ent(Swell = Big)

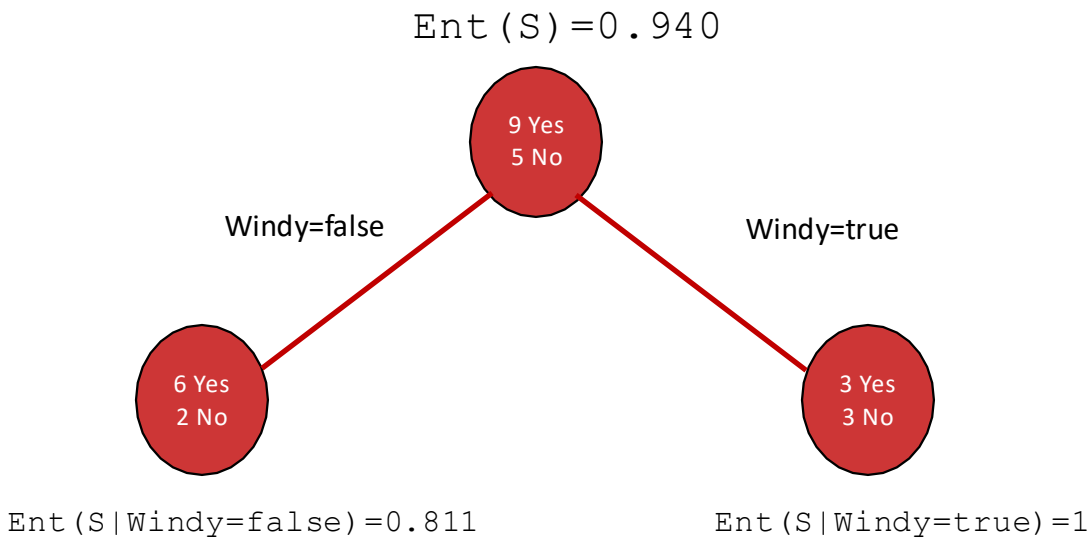
Entropy Example B

Ent(S)=0.940



Ent(S|Windy=false)=0.811

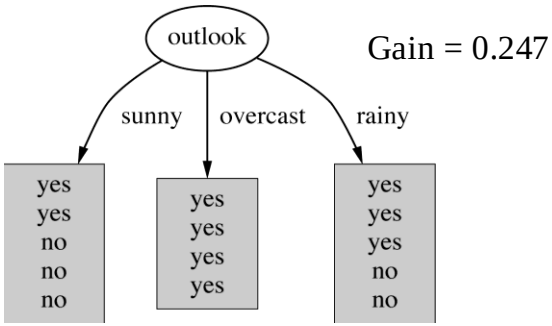
Ent(S|Windy=true)=1



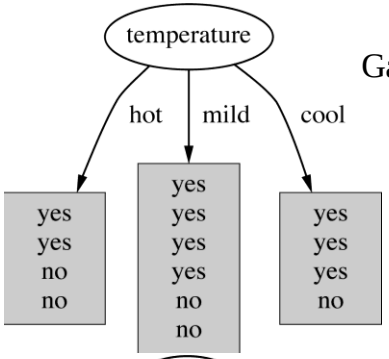
$$Gain\ (S, A) = Ent(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} Ent(S_v)$$

$$Gain\ (S, Windy) = Ent(S) - \sum_{v \in \{true, false\}} \frac{|S_v|}{|S|} Ent(S_v)$$

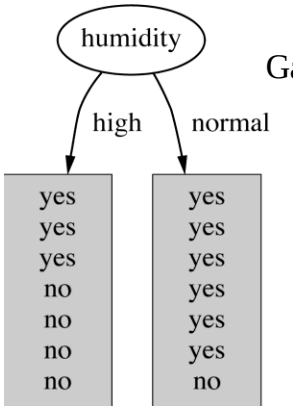
Information Gain Example



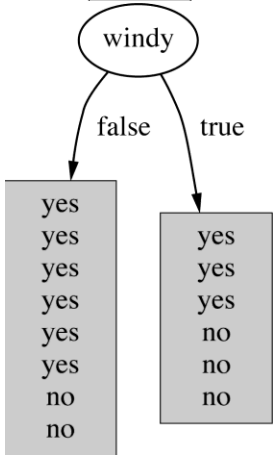
Gain = 0.247



Gain = 0.029



Gain = 0.152



Gain = 0.048

Entropy Tree

```
train_features, test_features, train_labels, test_labels
= model_selection.train_test_split(features, labels,
test_size=0.5, random_state=1)

dtc = tree.DecisionTreeClassifier(criterion = "entropy",
max_depth= 2, min_samples_split = 5)

dtc = dtc.fit(train_features, train_labels)

dtc.score(test_features, test_labels)
```

Print Tree

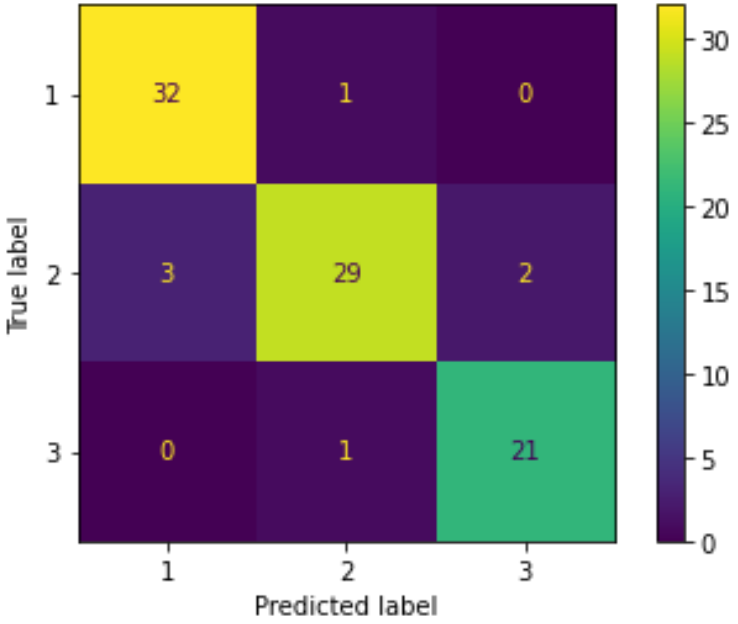
```
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 10)) # Resize figure
tree.plot_tree(dtc, feature_names = features.columns,
filled = True, rounded = True)
```

Confusion Matrix

```
import matplotlib.pyplot as plt
from sklearn import metrics

metrics.plot_confusion_matrix(dtc, test_features,
test_labels)
plt.show()
```

Confusion Matrix

```
import matplotlib.pyplot as plt
from sklearn import metrics

metrics.plot_confusion_matrix(dtc, test_features,
test_labels)
plt.show()
```



Hyperparameters search

```
from sklearn import tree
from sklearn import model_selection
import matplotlib.pyplot as plt

dtc = tree.DecisionTreeClassifier()
parameters = {?????}

train_features, test_features, train_labels, test_labels =
model_selection.train_test_split(features, labels, test_size=0.2,
random_state=0)


clf = model_selection.GridSearchCV(dtc, parameters, cv = 10)
clf.fit( train_features, train_labels)

print("The best classifier is:", clf.best_estimator_)
print("Its accuracy is:",clf.best_score_)
print("Its parameters are:",clf.best_params_)
```

Hyperparameters search

```
from sklearn import tree
from sklearn import model_selection
import matplotlib.pyplot as plt

dtc = tree.DecisionTreeClassifier()
parameters = {'criterion': ["entropy", "gini"], 'max_depth' : [2, 3, 4], 'min_impurity_decrease' : [0.01, 0.1, 0.2] }

train_features, test_features, train_labels, test_labels = model_selection.train_test_split(features, labels, test_size=0.2, random_state=0)


clf = model_selection.GridSearchCV(dtc, parameters, cv = 10)
clf.fit( train_features, train_labels)

print("The best classifier is:", clf.best_estimator_)
print("Its accuracy is:",clf.best_score_)
print("Its parameters are:",clf.best_params_)
```

Is it the final model accuracy?

Hyperparameters search

```
# The performances on the test set are:
clf.best_estimator_.score(test_features, test_labels)
```

That's all Folks!
SUPERB wallpapers